RAPPORT DE STAGE

LICENCE 3 MIASHS parcours MIAGE

ÉTUDE ET PROTOTYPAGE D'UN SYSTÈME DISTRIBUÉ POUR AMÉLIORER LA COORDINATION, LA RÉSILIENCE ET LA PERFORMANCE D'UN SERVICE NUMÉRIQUE AU SEIN D'I3S

ALEXANDRE BOUTRIK

15/06/2026 - 14/08/2026

Tuteur de l'apprenti : Monsieur Junior Dongo, assisté par Monsieur Luc Hogie
Tuteur académique : Monsieur Stéphane Tounsi

Etablissement / Formation : Université Côte d'Azur / DS4H
Entreprise d'accueil : Laboratoire I3S UMR 7271

SOMMAIRE

SOMMAIRE....2
REMERCIEMENTS....3
INTRODUCTION....4
PARTIE 1 - L'ENTREPRISE ET SON SECTEUR D'ACTIVITÉ....5
1.1 – PRÉSENTATION DE L'ENTREPRISE....5
1.2 – ANALYSE EXTERNE DE L'ENTREPRISE....7
PARTIE 2 - MON STAGE....9
2.1 - RÉSUMÉ DES MISSIONS....9
Gestion de Projet et Outils....10
2.2 - LE DÉTAILS DES MISSIONS....10
Projet Byransha et le Contexte du Besoin....10
Contexte des Applications Décentralisées....11
Framework Idawi....12
Confidentialité des Données Locale....13
Sécurisation des Communications....16
Autres Contributions....18
2.3 - LE BILAN....19
Résultats Obtenus....19
Difficultés Rencontrées et Solutions Apportées....19
Enseignements et Apports du Stage....21
CONCLUSION....22
GLOSSAIRE....23
BIBLIOGRAPHIE....25
ANNEXES....28
ANNEXE 1 : Jfabe, Architecture Centrale et Contrat de Base....29
ANNEXE 2 : Jfabe, Interface Client....30
ANNEXE 3 : Jfabe, Implémentation Cryptographique....31
ANNEXE 4 : Jfabe, Transformations Cryptographiques....32
ANNEXE 5 : Jfabe, Abstraction Mathématique....33
ANNEXE 6 : Jfabe, Analyse des politiques d'accès....34
ANNEXE 7 : Jfabe, Sérialisation des données....35
ANNEXE 8 : Jfabe, Exemple d'Utilisation de l'API....36

REMERCIEMENTS

Je tiens à adresser mes sincères remerciements à Monsieur Junior Dongo et Monsieur Luc Hogie pour m'avoir accordé l'opportunité de réaliser ce stage de recherche. Je souhaite remercier tout particulièrement Monsieur Luc Hogie pour son encadrement continu, son orientation et nos discussions constructives tout au long de cette période. Je tiens également à exprimer ma gratitude à mon tuteur académique, Monsieur Stephane Tounsi, pour avoir accepté d'encadrer ce travail ainsi que pour ses orientations tout au long de mon stage.

INTRODUCTION

J'ai pour ambition professionnelle de poursuivre une carrière de chercheur en cryptographie. Dès le début de mes études supérieures, j'ai toujours été particulièrement intéressé par l'environnement de la recherche. Avant de rejoindre la Licence MIAGE en France, j'étais inscrit dans cursus de Génie Informatique à l'Université Technologique Fédérale du Paraná (UTFPR), au Brésil. J'y ai eu l'opportunité de participer activement à plusieurs projets de recherche, notamment sous la direction du Professeur Alexandre Augusto Giron, docteur en Informatique, et ces travaux ont donné lieu à des publications (Boutrik et Giron, 2024 ; Schmitz et al., 2025). Ces premières immersions dans le monde scientifique m'ont permis de consolider ma passion pour les défis inhérents à la confidentialité et à la sécurisation de l'information. C'est par conséquent que j'ai ciblé un stage de recherche pour l'accomplissement de mon stage de fin d'études.

J'ai donc eu l'opportunité d'intégrer le Laboratoire I3S (UMR 7271), une structure de recherche d'excellence rattachée à l'Université Côte d'Azur. Mon stage s'est déroulé au sein du pôle COMRED (Laboratoire I3S, 2026b), et plus spécifiquement dans l'équipe Scale (Laboratoire I3S, 2026a). Cette équipe se concentre sur les problématiques complexes liées aux réseaux et aux systèmes distribués à grande échelle.

Sous la direction de mon tuteur de stage, Monsieur Junior Dongo, et avec l'assistance de Monsieur Luc Hogie, mes missions se sont inscrites dans le cadre de « l'étude et prototypage d'un système distribué pour améliorer la coordination, la résilience et la performance d'un service numérique au sein d'I3S ». Ce sujet s'aligne parfaitement avec mes intérêts, puisqu'il mêle habilement les thèmes d'architectures distribuées aux enjeux de la cryptographie.

Plus concrètement, ma mission s'est articulée autour d'Idawi (Hogie, 2025 ; Hogie, 2022a ; Hogie, 2022b), un *framework* de base de données décentralisé, ainsi que l'application Byransha (Hogie, 2026), qui seront exploré plus en détail dans la section 2 nommée « Mon Stage ».

PARTIE 1 - L'ENTREPRISE ET SON SECTEUR D'ACTIVITÉ

1.1 – PRÉSENTATION DE L'ENTREPRISE



Figure 1 : Logo du Laboratoire I3S.[1]

Le laboratoire d'Informatique, Signaux et Systèmes de Sophia Antipolis (i3S) a été officiellement fondé en 1989 à la suite de la fusion des laboratoires LISAN et LASSY. Il s'agit d'une unité mixte de recherche (UMR 7271) placée sous les tutelles principales du Centre National de la Recherche Scientifique (CNRS) et l'Université Côte d'Azur (UniCA), avec le Centre INRIA d'UniCA agissant en tant que tutelle secondaire. Dirigé par Frédéric Mallet depuis le mois de janvier 2022, le laboratoire i3s s'est imposé comme l'un des plus importants centres de recherche en technologie de l'information de la Côte d'Azur. Il bénéficie également d'un ancrage historique, ayant été l'un des tout premiers laboratoires à s'installer sur la technopole de Sophia Antipolis.

L'organisation rassemble près de 300 collaborateurs incarnant une grande diversité avec 36 nationalités représentées. Les effectifs permanents s'appuient sur 93 enseignants-chercheurs , 19 chercheurs rattachés au CNRS, 9 chercheurs affilies à Inria, ainsi que 19 membres dédiés aux fonctions de soutien et de support technique et administratif. A ce socle s'ajoutent 129 collaborateurs non permanents, parmi lesquels figurent plus d'une centaine de doctorants.

Le laboratoire s'appuie sur une structure financière solide totalisant 20 millions d'euros de ressources propres en 2023, lui permettant de générer 125 projets, dont 30 soutenus par l'Agence Nationale de la Recherche (ANR) et 8 projets européens de type H2020. Un organigramme détaillant l'ensemble de la structure hiérarchique et opérationnelle de l'établissement, ainsi qu'une matrice SWOT, est présenté ci-dessous.

[1] Source : Propriété du laboratoire I3S, extrait du site officiel : https://www.i3s.unice.fr .

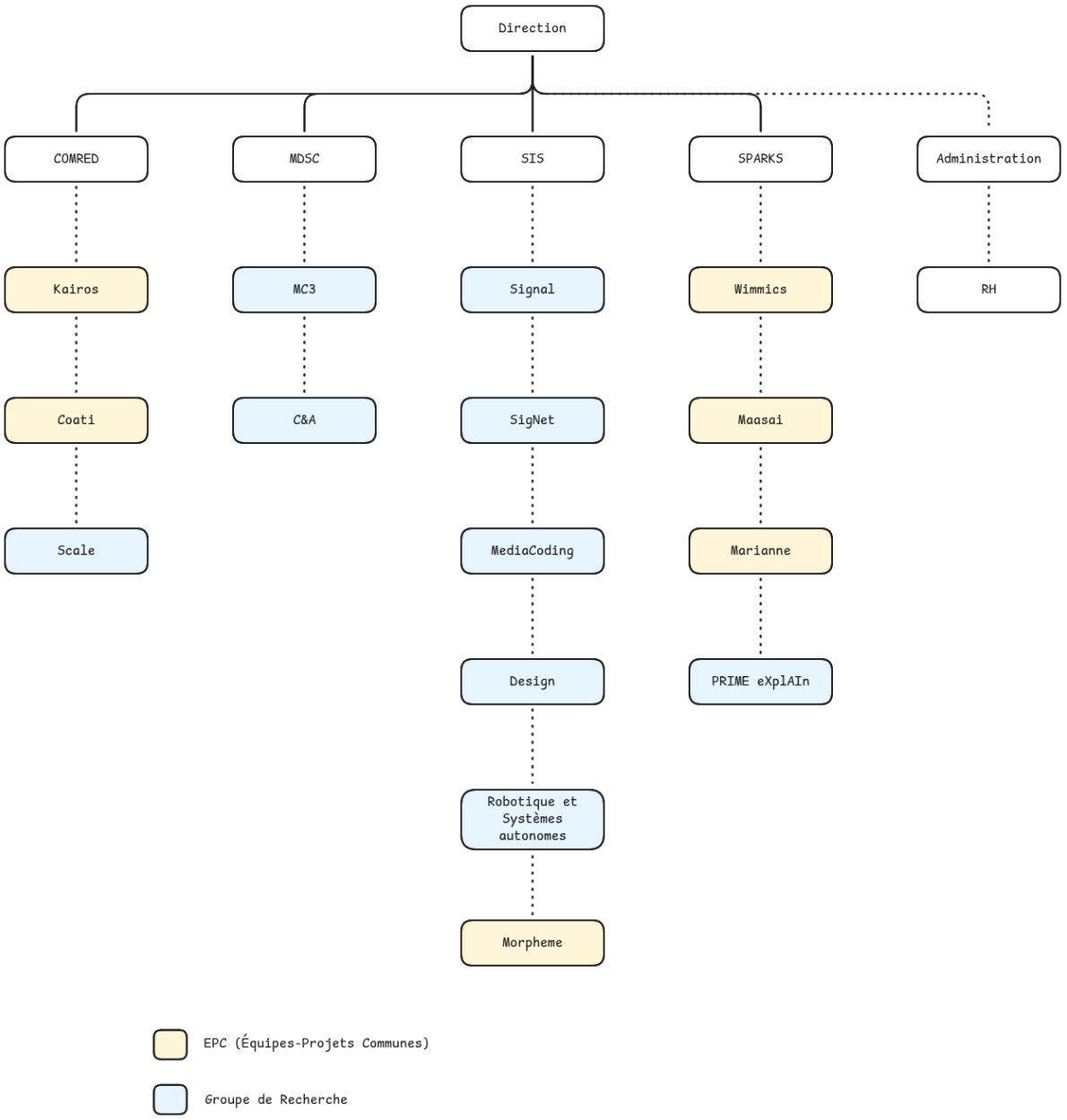


Figure 2 : Organigramme de la structure hiérarchique et opérationnelle de l'établissement.[2]

[2] Source : Réalisé par l'auteur à l'aide d'Excalidraw.

Forces	Faiblesses
Le laboratoire i3S bénéficie d'une structure financière solide et d'une réputation qui lui confèrent une forte attractivité auprès des chercheurs en quête de visibilité et de réputation scientifique. L'excellence de l'unité lui permet d'offrir un environnement propice au rayonnement de ces nombreux collaborateurs.	Le principal point de faiblesse du laboratoire réside dans sa fragmentation géographique. Faute de locaux d'accueil suffisants, les effectifs sont dispersés sur plusieurs sites distincts (I3S, Inria, Polytech). Cet éclatement physique limite les interactions et freine la communication interne, complexifiant ainsi la collaboration interdisciplinaire indispensable aux travaux de recherche.
Opportunités	**Menaces**
Le laboratoire a la formidable capacité de tirer parti de la fuite des cerveaux à l'échelle internationale en attirant sur le territoire français des talents scientifiques formés à l'étranger. L'alignement stratégique rigoureux sur les priorités politiques nationales et européennes constitue le levier principal pour capter les budgets massifs nécessaires à la recherche.	Le laboratoire fait face à un risque constant de fuite des cerveaux au niveau national, ce qui l'oblige à lutter sans relâche contre d'autres laboratoires concurrents pour maintenir son attractivité et retenir ses meilleurs éléments. Cette compétition s'inscrit dans un écosystème ou la survie dépend d'une course continue à la visibilité, aux publications et aux subventions.

Tableau 1 : Le SWOT du laboratoire i3S.

1.2 – ANALYSE EXTERNE DE L'ENTREPRISE

L'analyse externe du laboratoire i3S doit être appréhendée selon une grille de lecture différente de celle d'une entreprise commerciale classique. Dans cet environnement particulier, la concurrence ne s'exprime pas en termes de parts de marché. Elle se traduit plutôt par une course continue aux publications, la visibilité scientifique et la nécessité d'obtenir des financements. L'intégration des facteurs PESTEL permet de mettre en lumière les forces motrices de cet écosystème compétitif.

L'environnement politique et légal de l'unité est directement façonné par le cadre institutionnel de ces multiples tutelles, à savoir le CNRS, Université Côte d'Azur et Inria. Dans ce cadre, la collecte de budget revêt une dimension éminemment politique. Bien que les chercheurs aspirent naturellement à la liberté d'innovation, à la réputation et à la visibilité, la survie du laboratoire exige un alignement stratégique sur les feuilles de route nationales et européennes. Le recrutement des doctorants et l'allocation des bourses dépendent également de ces mêmes orientations politiques. Le laboratoire se

trouve donc face à un défi d'équilibre : il s'agit de concilier la volonté d'indépendance des équipes avec la nécessité vitale de se conformer aux priorités étatiques pour sécuriser les budgets de fonctionnement.

L'environnement économique repose sur un modèle de financement hautement concurrentiel. La captation de subventions ciblées, illustrée par les 30 projets ANR et les 8 projets européens H2020 actuellement gérés, constitue la pérennité opérationnelle de l'unité. Cette dépendance économique limite la prise de risque sur des thématiques indépendantes et contraint le laboratoire à justifier continuellement la rentabilité scientifique de ses travaux auprès des bailleurs de fonds. Toutefois, ce modèle financier est stratégiquement complété par la valorisation directe de la recherche. Le laboratoire constitue un levier de création de richesse grâce à la propriété intellectuelle produite par ses chercheurs et doctorants, notamment par le dépôt de nombreux brevets. Cet écosystème économique est activement dynamisé par le transfert de technologies vers le secteur privé, permettant la création de startups innovantes (telles que PearCode (Pearcode, 2026) et EyeNav (EYENAV Robotics, 2026)), et allant jusqu'à la revente stratégique d'entreprises à succès issues du laboratoire (comme Cintoo (Cintoo, 2026) et Pixamp (Laboratoire I3S, 2026c)).

L'environnement socioculturel est lié à l'attractivité des talents, ce qui soulève la question de la fuite des cerveaux (*brain drain*). Cette dynamique s'opère à deux niveaux distincts. A l'échelle nationale, le laboratoire fait face à un risque constant de départ de ses propres chercheurs. Il doit lutter contre d'autres pôles d'excellence concurrents sur le territoire français pour maintenir son attractivité, offrir des conditions de recherche optimales et retenir ses meilleurs éléments. Cependant, à l'échelle internationale, la dynamique devient une opportunité stratégique. Fort de ses 36 nationalités représentées, I3S capitalise aussi sur l'importation de compétences scientifiques formées à l'étranger.

En définitive, le laboratoire I3S évolue dans un écosystème complexe où il doit perpétuellement équilibrer la quête de son indépendance intellectuelle avec la réalité d'une compétition pour les financements institutionnels, la monétisation de son savoir-faire et la captation des meilleurs cerveaux.

PARTIE 2 - MON STAGE

2.1 - RÉSUMÉ DES MISSIONS

Le stage, d'une durée de deux mois, s'est déroulé au sein de l'équipe Scale du laboratoire I3S. Les missions qui m'ont été confiées concernent le projet Idawi, un *framework* d'applications décentralisées conçu pour prendre en charge le stockage de données, la communication, la sécurité ainsi que les interfaces utilisateur et programmatives (APIs). Deux axes de travail ont structuré mon intervention : la confidentialité de données stockées localement et la sécurisation des communications entre les participants du réseau décentralisée. La répartition des tâches sur l'ensemble du stage est synthétisée dans le diagramme de Gantt présenté ci-dessous.

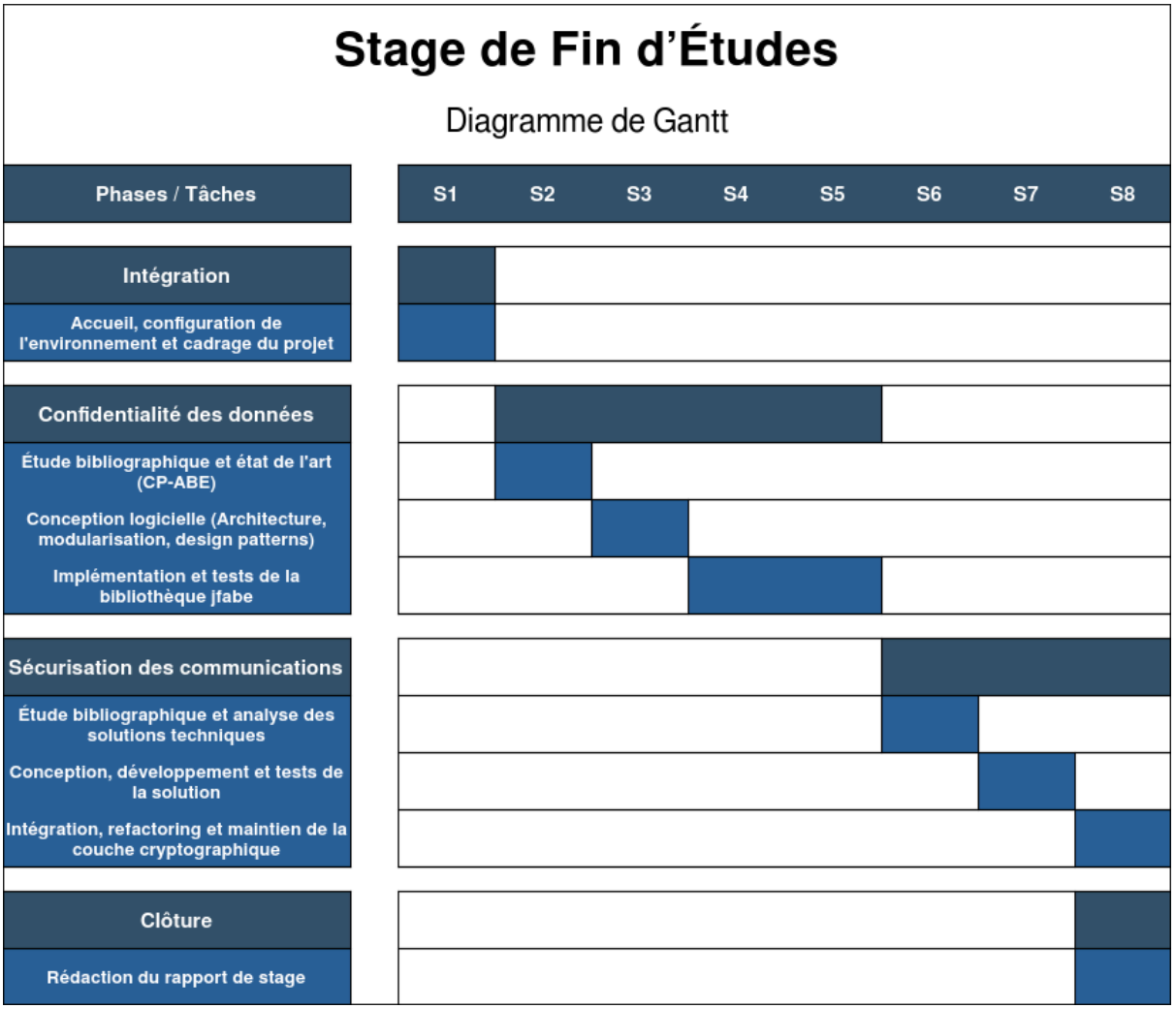


Figure 3 : Planification et déroulement du stage.[3]

[3] Source : Réalisé par l'auteur via LibreOffice Calc.

Gestion de Projet et Outils

Pour la réalisation du projet, l'équipe a adopté une organisation fluide reposant sur une communication continue en présentiel. Étant tous installés dans la même pièce, nous avons pu aborder les problèmes directement et de vive voix, en nous appuyant parfois sur un tableau blanc pour schématiser nos réflexions. Cette proximité spatiale a permis une vitesse de développement remarquable, qui aurait été tout simplement impossible dans un cadre de travail à distance.

Mon quotidien s'articulait autour d'une boucle de travail itérative. Je débutais généralement par une phase de recherche et de lecture bibliographique, suivie par une discussion avec Monsieur Hogie pour recueillir ses retours et confronter nos idées, avant de retourner à mes recherches avec de nouvelles pistes à explorer.

Sur le plan technique, la collaboration et le suivi des versions ont été assurés au moyen de git (Torvalds et al., 2005) ainsi qu'un déploiement SSH vers un environnement de *staging*. Concernant les environnements de développement, aucune restriction n'a été imposée, laissant à chaque membre le soin d'utiliser l'outil avec lequel il se sentait le plus à l'aise. Ainsi, Monsieur Hogie développait sur Eclipse (Eclipse Foundation, 2001) pendant que j'ai utilisé Neovim (Neovim contributors, 2014), et Monsieur Malaussena, un autre stagiaire de l'équipe, travaillait sur l'aspect IA du projet avec Visual Studio Code (Microsoft, 2015).

2.2 - LE DÉTAILS DES MISSIONS

Projet Byransha et le Contexte du Besoin

Le développement du *framework* Idawi s'inscrit dans une démarche scientifique portée par Monsieur Luc Hogie, membre du pôle COMRED, constitué des équipes Coati, Kairos, Scale et Signet, qui a d'ailleurs consacré 4 publications dans des conférences et workshops internationaux sur le sujet depuis 2022. Parallèlement à ces travaux, le service des ressources humaines (RH) du laboratoire I3S a exprimé la nécessité de se doter d'un nouveau logiciel pour moderniser et optimiser sa gestion interne. C'est pour répondre précisément à cette demande qu'a été initié le projet Byransha.

Saisissant l'opportunité de lier ce besoin fonctionnel à des travaux scientifiques, Monsieur Hogie a donc choisi de concevoir cette solution en y intégrant une dimension de recherche. L'architecture globale du projet repose ainsi sur une synergie : Idawi fournit le socle technologique fondamental, tandis que Byransha agit comme l'application métier bâtie sur cette fondation. Le logiciel RH devient alors le cas d'usage pour éprouver, valider et exploiter pleinement les capacités du *framework* Idawi en conditions réelles de production.

Le langage Java a été retenu pour sa portabilité multiplateforme et pour l'homogénéité qu'il autorise. Le projet est en effet développé en Java pur, ce qui permet de livrer l'application sous la forme d'un unique fichier exécutable .jar. Ce fichier peut être exécuté sur toute machine disposant d'une machine virtuelle Java (JVM), que ce soit sous Windows, Linux ou MacOS. Ainsi, le service RH peut utiliser librement les

postes de son choix, sans que l'équipe de développement ait à porter ou à maintenir plusieurs versions spécifiques à chaque système d'exploitation. Cette unification simplifie également la maintenance logicielle en comparaison d'une architecture classique ou le *front-end* est généralement écrit en JavaScript tandis que le *back-end* mobilise une technologie distincte.

Contexte des Applications Décentralisées

Avant de procéder, il est utile de préciser ce qui distingue fondamentalement une application décentralisée d'une application centralisée, ainsi que les forces et faiblesses propres à chaque modèle.

Dans une architecture centralisée, l'ensemble des données et des traitements transite par un serveur unique, ou un nombre de serveurs placés sous le contrôle d'une même entité. Cette organisation offre une grande simplicité de gestion et facilite le maintien de la cohérence des informations. En contrepartie, le serveur central constitue un point de défaillance critique : une panne, une attaque ou une décision unilatérale de l'opérateur peut interrompre l'intégralité du service. De plus, ce modèle confère à l'autorité centrale un pouvoir important sur les données des utilisateurs, ce qui soulève des interrogations en matière de censure ou même de corruption.

Une application décentralisée repose au contraire sur un réseau de participants équivalents qui coopèrent sans qu'aucun d'entre eux ne détienne l'autorité absolue sur l'ensemble du système. Les données sont répliquées ou partitionnées entre les participants, et les décisions sont prises collectivement selon des règles préétablies. Ce paradigme offre plusieurs avantages. En supprimant le point central de défaillance, il améliore la résilience : le réseau continue de fonctionner même si certains participants deviennent inaccessibles. Il renforce également l'autonomie des utilisateurs, qui conservent la maîtrise de leurs données, sans dépendre d'un tiers de confiance. Enfin, il peut favoriser la transparence et l'audibilité, chaque participant pouvant vérifier le respect des règles.

Toutefois, ces bénéfices s'accompagnent de défis techniques substantiels. L'absence d'autorité centrale rend la synchronisation des états plus complexe. Les mécanismes de consensus ou de réplication introduisent une latence et une charge supplémentaires, qui peuvent nuire aux performances. De surcroît, la gouvernance d'un système distribué est délicate, car il est difficile de faire évoluer le protocole sans coordination centralisée.

Le schéma ci-dessous illustre la différence structurelle entre les deux modèles : d'un côté un unique serveur central autour duquel gravitent les clients, de l'autre un maillage de pairs interconnectés partageant directement les informations.

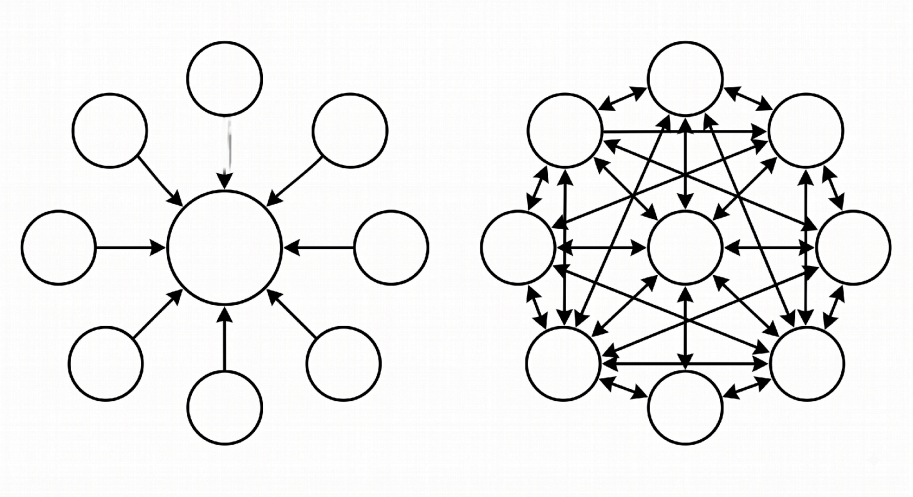


Figure 4 : Modèle centralisé et modèle décentralisé.[4]

Framework Idawi

Afin d'appréhender le fonctionnement de Idawi, il convient de clarifier la terminologie adoptée par le projet. Au sein de cette architecture, chaque participant ou nœud du réseau est désigné sous le terme de « participant ». La base de données, quant à elle, est structurée sous la forme d'un graphe constitué des « éléments ». Conceptuellement, ces éléments représentent des caractéristiques précises ou des classes Java, telles que « Person », « String » ou encore « Name ».

Une particularité fondamentale de Idawi est le fait que chaque participant du réseau héberge localement une réplique complète de la base de données orientée graphe. Comme le montre l'illustration ci-dessus, cette base n'est pas stockée directement sous la forme de graphe : elle est en réalité conservée physiquement sur le disque sous la forme d'une liste d'événements. C'est au moment de l'exécution de l'application que cette succession d'évènements est lue et interprétée directement dans la mémoire (RAM), permettant ainsi de reconstruire dynamiquement le graphe de éléments exploitable par l'interface utilisateur. Enfin, pour garantir une cohérence des informations entre tous les participants, ce sont ces mêmes événements qui sont diffusés et échangés sur le réseau, constituant ainsi le mécanisme de synchronisation de l'application. L'image ci-dessous illustre le modèle de données de Idawi.

[4] Source : Réalisé par l'auteur à l'aide d'Excalidraw.

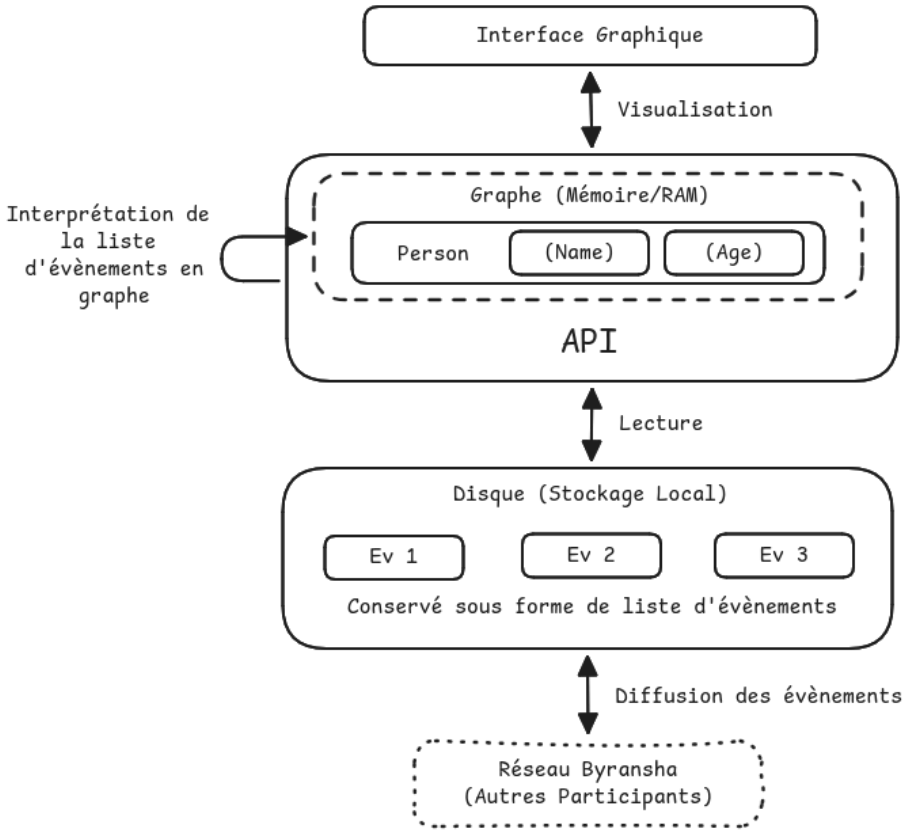


Figure 5 : Architecture de données de Idawi.[5]

Dans ce cadre, mes missions de stage sont articulées autour de deux enjeux principaux : la confidentialité des données stockées localement et la sécurisation des communications entre les participants.

Confidentialité des Données Locale

Le premier axe de mon stage concerne la confidentialité des données. Puisque la base de données est répliquée chez tous les participants, le contrôle d'accès et la confidentialité des informations soulèvent des défis techniques majeurs. Si la liste d'événements sont stockées en clair, chaque participant a techniquement la capacité de lire la totalité des données, contournant et violant ainsi les niveaux de confidentialité requis par l'application. Par conséquent, la gestion de cette confidentialité et l'application d'un contrôle d'accès doivent impérativement s'opérer localement et de manière hors ligne, directement sur la machine de chaque participant, afin de respecter l'aspect décentralisé de l'architecture de Idawi.

Pour répondre à cette problématique, la solution cryptographique retenue s'appuie sur le chiffrement basé sur les attributs (*Attribute-Based Encryption*), un concept initié par Goyal et al. en 2006. Plus précisément, nous appliquons la variante CP-ABE (*Ciphertext-Policy Attribute-Based Encryption*), introduite initialement par Bethencourt et al. en 2007. Cette solution permet de chiffrer un message en définissant une politique d'accès précise qui combine différents attributs au moyen des opérateurs

[5] Source : Réalisé par l'auteur à l'aide d'Excalidraw.

logiques AND et OR. A titre d'exemple, un texte chiffré sous la politique d'accès « ABC and DEF » ne pourra être déchiffré et lu que par un participant possédant une clé cryptographique qui intègre simultanément l'attribut ABC et l'attribut DEF.

Afin d'intégrer concrètement cette solution à Idawi, j'ai développé et finalisé une bibliothèque codée entièrement en Java nommée jfabe (Boutrik, 2026). Au sein de cette bibliothèque, notre choix d'implémentation pour Idawi s'est porté sur le schéma FABEO (*Fast Attribute-Based Encryption with Optimal Security*), proposé par Riepel et Wee en 2022. Ce schéma succède au modèle FAME (Agrawal et Chase, 2017) en apportant des performances significativement accrues, notamment grâce à des textes chiffres plus courts et à un processus de déchiffrement plus rapide. De surcroît, le schéma FABEO a été sélectionné par l'équipe car sa preuve de sécurité repose sur un modèle multi-défi. Cette propriété garantit que le niveau de sécurité global ne se dégrade pas face à l'immense quantité de textes chiffres que chaque participant doit stocker localement pour maintenir une copie complète de la base de données.

Comme l'illustre la figure ci-dessous, le code source de la bibliothèque est structuré en plusieurs paquetages distincts : api, core, schemes, transform, pairing, policy et serialization, interagissant de manière découplée. L'ensemble de choix architecturaux spécifiques à chaque paquetage, ainsi que les patrons de conception associés, sont détaillés plus en profondeur dans les Annexes 1 à 7. De plus, un exemple d'implémentation côte utilisateur de l'API jfabe est fourni en Annexe 8.

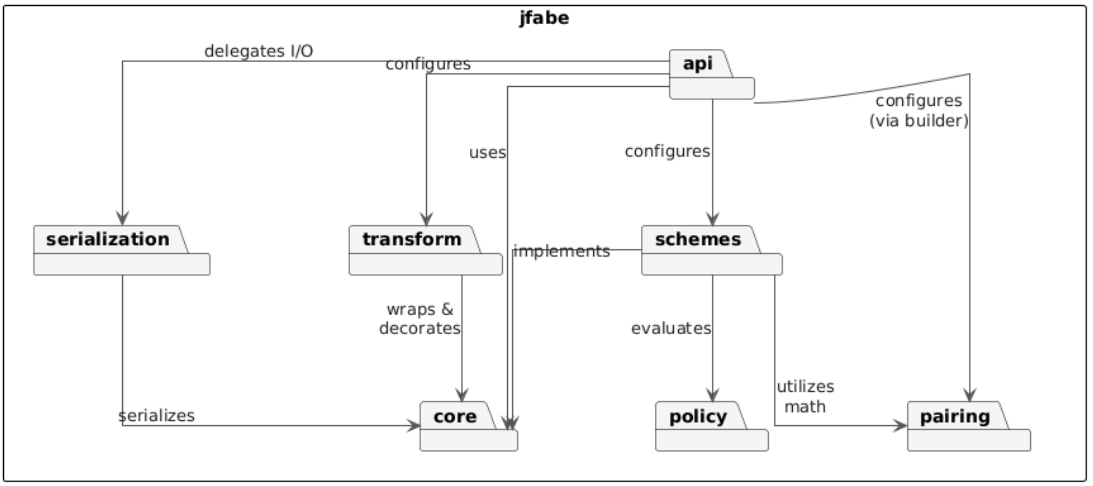


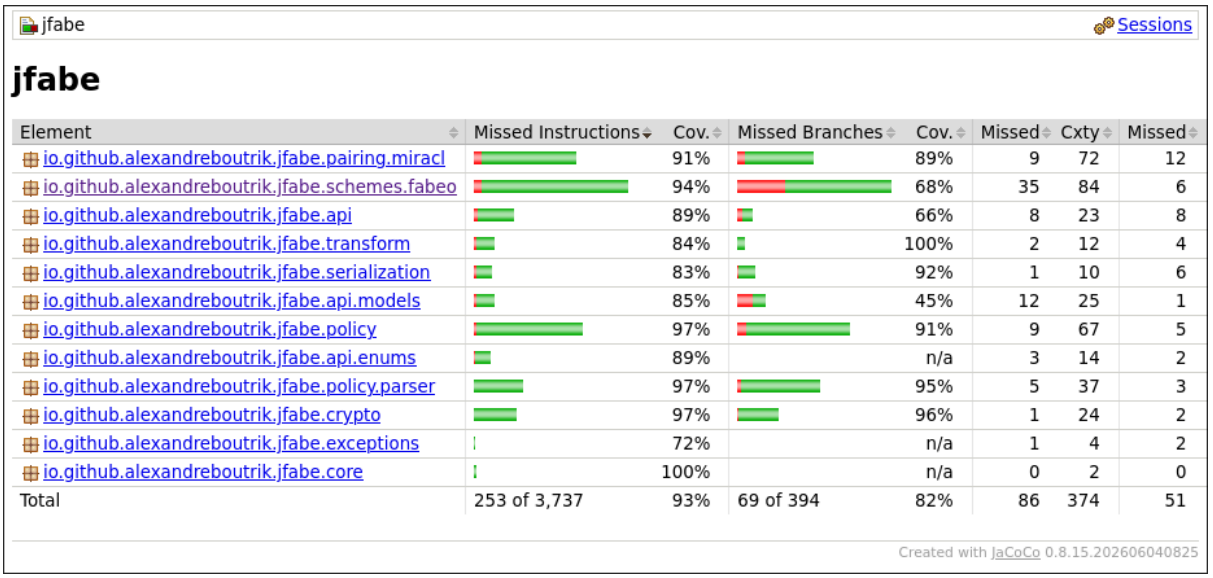
Figure 6 : Diagramme d'architecture illustrant les interactions entre les paquetages de jfabe.[6]

L'architecture logicielle de la bibliothèque jfabe a été conçue pour offrir une extensibilité maximale, s'organisant autour des composants interchangeables : les schémas (*Scheme*), les transformations (*Transform*) et les fonctionnalités additionnelles (*Feature*). Il est ainsi possible d'atteindre un niveau de sécurité élevé contre les attaques à chiffré choisi (*Chosen Ciphertext Attacks*) (Bellare et al., 1998) en appliquant une transformation de Fujisaki-Okamoto (Hofheinz et al., 2017), désignée par Transform.FO_KEM_AEAD. Bien qu'aucune fonctionnalité supplémentaire ne soit disponible dans la

[6] Source : Généré par l'auteur via PlantUML. Le code source est consultable à l'adresse suivante : https://github.com/alexandreboutrik/jfabe/blob/main/docs/diagrams/jfabe.

version de production actuelle, l'ajout d'une option de traçabilité (Liu et al., 2018), est déjà planifié pour une itération future de la bibliothèque. Cette conception fondamentalement modulaire de jfabe assure également la pérennité du *framework* en facilitant la transition éventuelle vers des schémas cryptographiques post-quantiques en accord avec les directives de l'ANSSI fixées pour l'horizon 2030 (ANSSI, 2026).

Les tests unitaires ont fait l'objet d'une attention particulière tout au long du développement de jfabe. La couverture des instructions (Instruction Coverage, ou IC) s'élève à 93 %, ce qui confirme que la majorité des lignes de code du projet sont exercées par les tests, tandis que la couverture des décisions (Decision Coverage, ou DC) atteint un taux satisfaisant de 82 %, garantissant ainsi l'évaluation des différentes branches conditionnelles du code. Ces résultats, générés par le plugin JaCoCo (EclEmma team, 2019) et présentes dans la figure ci-dessous, attestent de la robustesse de la bibliothèque et de la fiabilité des mécanismes cryptographiques implémentés.


jfabe

Sessions

jfabe

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed
io.github.alexandreboutrik.jfabe.pairing.miracl		91%		89%	9	72	12
io.github.alexandreboutrik.jfabe.schemes.fabeo		94%		68%	35	84	6
io.github.alexandreboutrik.jfabe.api		89%		66%	8	23	8
io.github.alexandreboutrik.jfabe.transform		84%		100%	2	12	4
io.github.alexandreboutrik.jfabe.serialization		83%		92%	1	10	6
io.github.alexandreboutrik.jfabe.api.models		85%		45%	12	25	1
io.github.alexandreboutrik.jfabe.policy		97%		91%	9	67	5
io.github.alexandreboutrik.jfabe.api.enums		89%		n/a	3	14	2
io.github.alexandreboutrik.jfabe.policy.parser		97%		95%	5	37	3
io.github.alexandreboutrik.jfabe.crypto		97%		96%	1	24	2
io.github.alexandreboutrik.jfabe.exceptions		72%		n/a	1	4	2
io.github.alexandreboutrik.jfabe.core		100%		n/a	0	2	0
Total	253 of 3,737	93%	69 of 394	82%	86	374	51

Created with JaCoCo 0.8.15.202606040825

Figure 7 : Rapport d'analyse de JaCoCo illustrant la couverture de tests de jfabe.[7]

Bien que la bibliothèque jfabe soit d'ores et déjà finalisée et pleinement opérationnelle, des contraintes de temps ont conduit l'équipe à différer son intégration au sein de Idawi. Ainsi, la première version du *framework* pourra stocker la liste d'événements en clair sur chaque machine, ou la retélécharger dans son intégralité à chaque démarrage. Cela permettrait d'éviter l'accès aux données via le disque local. Par conséquent, le chiffrement basé sur les attributs et les mécanismes de contrôle d'accès qu'il permet ne seront activés que lors de la seconde publication. Ce choix délibéré permet de livrer rapidement un MVP (*Minimum Viable Product*) tout en réservant une montée progressive des garanties de sécurité.

[7] Source : Capture d'écran générée par l'auteur via le plugin JaCoCo.

Sécurisation des Communications

Le second axe de mes missions a porté sur la sécurisation des échanges au sein du réseau décentralisé. Deux problématiques majeures se posent : déterminer qui est autorisé à rejoindre le réseau, et protéger les messages contre l'interception durant leur transit.

Pour répondre à la question de l'autorisation, une approche s'inspirant du modèle « Known Known » (KK), tel qu'on le retrouve dans le Noise Protocol Framework (Perrin, 2018), a été retenue. Un candidat souhaitant intégrer le réseau commence par générer localement sa propre paire de clés publique et privée, puis il transmet sa clé publique à un participant déjà présent par un canal hors réseau. Ce participant ajoute alors manuellement cette clé a sa liste de pairs de confiance, ce qui permet au nouveau venu d'établir une communication sécurisée avec lui.

La confiance s'étend progressivement par transitivité au sein du graphe du réseau. Le schéma ci-dessus illustre ce mécanisme dans le cas où le participant B ajoute la clé publique de A, autorisant ainsi A à interagir avec B puis, par rebond, avec les autres membres du réseau connectés à B (C et D) et ainsi de suite.

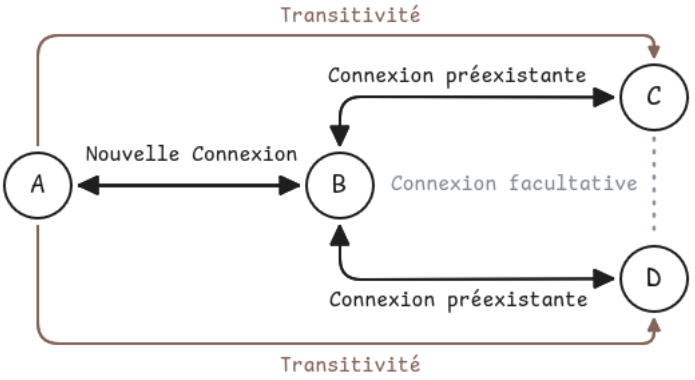


Figure 8 : Début d'une connexion sur réseau Idawi.[8]

Afin de pallier les difficultés de connectivité, notamment lorsqu'un destinataire se trouve derrière un pare-feu, un système de relais a été implémenté. Si A possède la clé publique de C mais ne peut l'atteindre directement, alors que C est connecté en tant que voisin de B et que B est lui-même voisin de A, A peut transmettre son message à C en le faisant transiter par B. Le terme de « voisin » désigne ici une connexion directe dans le graphe de recouvrement du réseau.

Sur le plan cryptographique, le protocole adopté s'appuie sur la bibliothèque NaCl (Bernstein et al., 2011). La primitive CryptoBox met en œuvre un chiffrement asymétrique permettant à deux participants de dériver localement une clé symétrique partagée, utilisée ensuite pour chiffrer les messages. Comme l'opération la plus coûteuse est la génération de cette clé partagée, NaCl propose également la primitive SecretBox, qui se contente de prendre une clé symétrique partagée déjà établie et de chiffrer les données. Cette distinction permet une optimisation en cache : lorsqu'un même destinataire est

[8] Source : Réalisé par l'auteur à l'aide d'Excalidraw.

sollicité de façon répétée, la clé partagée préalablement calculée peut être reutilisée, évitant ainsi de lourds calculs pour chaque envoi.

Idawi centralise les opérations de chiffrement dans le fichier security/NetworkBox.java. Trois méthodes ont été définies en conséquence. La méthode encrypt() correspond à CryptoBox ; la méthode agreeOnSharedSecret() assure la génération de la clé secrète partagée ; et enfin, la méthode encryptFast() réalise le chiffrement rapide au moyen de SecretBox. Ces primitives constituent le socle des échanges sécurisés.

La communication sur Idawi débute lorsqu'un participant, nommé A, se connecte à ses voisins. Il génère alors une paire de clés éphémères, puis diffuse à ces voisins un message contenant son identité, la clé publique éphémère ainsi qu'une signature produite avec sa clé privée. L'emploi de clés éphémères est indispensable pour garantir la propriété de secret parfait (*Forward Secrecy*) (Diffie et al., 1992 : 107-125) : en cas de compromission ultérieure de la clé privée d'un participant, les échanges antérieures, chiffres au moyen de clés éphémères distinctes, demeurent inaccessibles. La clé permanente ne sert donc qu'à signer la clé éphémère et à en attester l'authenticité auprès des autres participants.

Dès réception de ce message initial, les voisins de A le réémettent à l'ensemble du réseau, puis répondent à A en lui adressant la liste des clés publiques éphémères dont ils ont connaissance. A parcourt alors cette liste, vérifie les signatures pour les clés dont il possède la clé publique, et classe ces entrées comme dignes de confiance ; les autres sont simplement conservées en vue d'une éventuelle retransmission ultérieure à de nouveaux voisins. L'image ci-dessous illustre ce processus pour un enchainement A -> B -> C : A envoie sa clé éphémère à B et reçoit de la part de B une liste contenant les clés éphémères de B et de C, tandis que B relaie la clé éphémère de A à C.

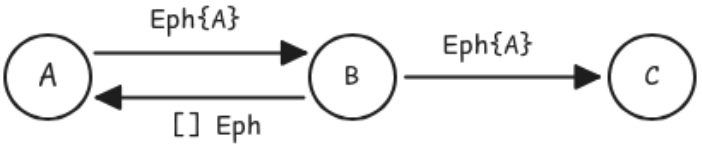


Figure 9 : Échange des clés éphémères.[9]

Une fois cette phase de découverte des clés éphémères achevée, la transmission des événements proprement dits peut commencer. Chaque message de Idawi est structuré en deux couches. La charge utile de bout en bout, destinée au seul destinataire final, est une structure E2EPayload contenant une estampille temporelle, le corps du message et l'identifiant de l'émetteur. Cet objet sérialisé est chiffré en suite avec SecretBox au moyen de la clé symétrique partagée entre l'émetteur et le destinataire final, produisant ainsi le champ E2Ebytes. Ce dernier est à son tour encapsulé dans une structure HopPayload qui inclut le chemin parcouru (la liste des identifiants de participants du relais traverses) et E2Ebytes.

[9] Source : Réalisé par l'auteur à l'aide d'Excalidraw.

On applique donc deux couches de chiffrement successives : l'une, externe, déchiffrée par chaque saut, et l'autre, interne, que seul le destinataire final peut déchiffrer pour accéder au contenu original. L'image ci-dessous illustre ce schéma.

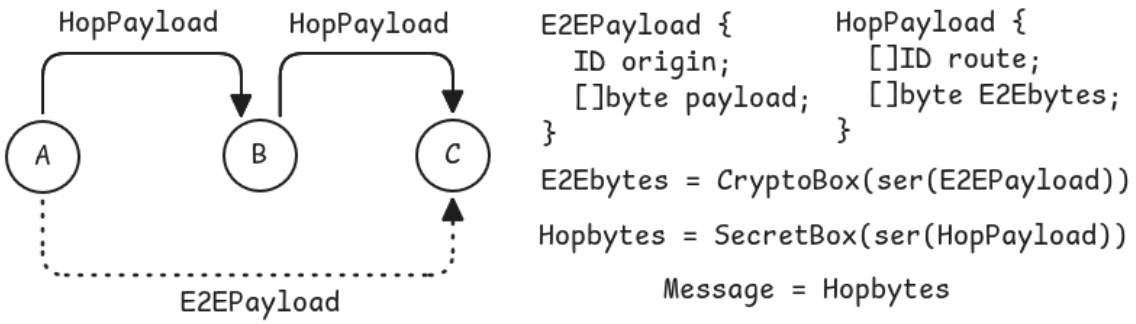


Figure 10 : Chiffrement en couches de Idawi.[10]

Il est à noter que, contrairement au premier axe de mon stage dont l'intégration a été repoussée, le dispositif de sécurisation des communications décrit dans cette sous-section est d'ores et déjà implémenté et fera partie de la première version de Idawi. Les échanges entre participants bénéficient donc, dès cette livraison initiale, des propriétés de confidentialité, d'intégrité et d'authentification apportées par le double chiffrement et le mécanisme de relais. La seconde publication du *framework* viendra renforcer cette architecture, notamment en introduisant des signatures imbriquées dans le champ de routage de la structure HopPayload, ce qui permettra de garantir l'authenticité du chemin emprunté par chaque message jusqu'à son destinataire.

Autres Contributions

Outre mes responsabilités principales touchant à la confidentialité des données locales et la sécurisation des communications, j'ai également contribué à d'autres dimensions du projet. J'ai notamment pris part aux discussions relatives au routage et à la topologie du réseau, en aidant l'équipe à concevoir ou à améliorer des solutions face aux problèmes qui émergeaient naturellement durant la conceptualisation du *framework*.

La contribution que je tiens à souligner porte sur la stratégie d'identification des éléments du graphe, une problématique mêlant l'entropie et la cryptographie. L'architecture générale a été essentiellement pensée par Monsieur Hogie, et j'ai émis des recommandations concernant le choix de la mise en place d'une fonction de hachage, en insistant sur la nécessité de garantir une résistance contre les collisions et à l'irréversibilité du procédé, afin d'empêcher un participant malveillant d'injecter les données d'un élément en menant une attaque de préimage.

Ce schéma d'identification a été introduit pour résoudre une difficulté précise : supposons qu'un attribut S arrive à destination avant son parent P en raison d'un problème de transmission : il faut alors que le destinataire puisse instancier S puis le rattacher à P lorsque ce dernier est finalement reçu. La solution retenue consiste à doter P d'un identifiant produit par un générateur de nombres aléatoires cryptographiquement sûr *(Cryptographic Secure Random Number Generator*, ou CSRNG) (Paar et al.,

[10] Source : Réalisé par l'auteur à l'aide d'Excalidraw.

2024 : 44), ce qui rend à la fois imprévisible et infalsifiable, et à définir l'identifiant de S comme le hachage Keccak (Bertoni et al., 2011) de la concaténation de l'identifiant de P et du nom de la classe de S, comme démontré dans les formules ci-dessous.

$$P_{id} = \text{CSRNG}() \qquad S_{id} = \text{Keccak}(P_{id} + \text{"S"})$$

Ce mécanisme assure que l'identifiant de P est non-déterministe et que l'identifiant de S est déterministe qu'à partir du moment où l'on connaît celui de P, tout en préservant l'intégrité structurelle du graphe et interdisant ainsi toute compromission par attaque de préimage.

2.3 - LE BILAN

Résultats Obtenus

Le premier résultat concret est la livraison de la bibliothèque jfabe, qui a été finalisée dans le respect de contraintes de temps et des exigences de sécurité. Le second résultat est la mise en place du mécanisme de sécurisation des communications reposant sur NaCl et sur le double chiffrement des messages, déjà intégré dès la première version du *framework*.

Monsieur Hogie, leader de l'équipe, a souligné ma capacité à monter rapidement en compétences sur des sujets cryptographiques et à proposer des solutions adaptées aux problèmes rencontrés. Il a également noté mon autonomie dans le développement, ma capacité à travailler sur des problèmes variés, et ma rigueur dans la prise en compte des impacts sur la sécurité, ainsi que ma participation active aux discussions techniques de l'équipe. Je juge la gestion du temps comme satisfaisante, les priorités ayant été définies et les livrables produits dans les délais convenus. Ces appréciations me confortent dans l'idée que ce stage a été une expérience formatrice.

Difficultés Rencontrées et Solutions Apportées

Lors de la réalisation du stage, plusieurs difficultés ont jalonné le développement des solutions de sécurité destinées à Idawi.

La première d'entre elles a trait à la gestion de la révocation des clés dans le chiffrement basé sur les attributs (ABE). Dans un schéma CP-ABE classique, une fois qu'une clé est distribuée à un participant, il est difficile de la révoquer sans devoir rechiffrer l'intégralité de la base de données ou distribuer de nouvelles clés à tous les autres participants. Pour contourner cette limitation, nous avons discuté un système d'époques (*epoch*) : les clés d'attributs sont émises avec une durée de validité limitée et doivent être renouvelées périodiquement. Cela permet de retirer un attribut d'un participant simplement en ne lui délivrant pas la clé correspondante pour l'époque suivante, sans affecter les autres participants du réseau. Cependant, cette approche ne résout pas entièrement le problème temporel : un participant qui ne participe pas à l'époque suivante peut encore déchiffrer les données qui ont été chiffrées au cours de l'époque précédente. Cette difficulté est complexe à résoudre, car la base de données étant répliquée chez tous les participants, il est impossible de les contraindre à effacer localement les anciennes données lors du passage à une nouvelle époque.

Une deuxième difficulté a trait au développement de la bibliothèque jfabe elle-même. La mise en œuvre du chiffrement CP-ABE nécessitait en effet une dépendance cryptographique à base de couplages (Pairings-Based Cryptography) (Galbraith et al., 2008). Dans un premier temps, je me suis appuyé sur la bibliothèque jPBC (Java Pairing-Based Cryptography Library) (De Caro et Iovino, 2011), mais celle-ci n'est plus maintenue depuis 2013 et n'a jamais intégré de support pour les courbes BLS modernes. De surcroît, l'attaque exTNFS (Extended Tower Number Field Sieve) (Kim & Barbulescu, 2015) a réduit le niveau de sécurité effectif de la courbe BN254, utilisée par jPBC, de 128 bits a environ 100 bits, ce qui impose, pour conserver 128 avec jPBC, l'utilisation de la courbe BN462, environ 6x plus lente que BN254. Il a donc fallu identifier une bibliothèque de remplacement répondant à l'ensemble de contraintes du projet. Cette recherche s'est révélée délicate car toutes les bibliothèques de couplages n'implémentent pas forcément la sérialisation du groupe Gt (zkcrypto contributors, 2020), fonctionnalité pourtant indispensable dans les schémas ou le texte chiffré est un élément de Gt. J'ai finalement retenu MiraCL Core (MIRACL, 2026), une bibliothèque activement maintenue, qui satisfait ces exigences et prend nativement en charge la courbe moderne et largement adoptée BLS12-381 (Bowe, 2017).

Une troisième difficulté est apparue lors de l'évaluation initiale du protocole de communication. J'avais envisagé de s'appuyer sur le Noise Protocol Framework, reconnu pour sa flexibilité et sa robustesse cryptographique. Cependant, une contrainte majeure s'est posée : dans Noise, les schémas d'échange à sens unique (non interactifs) limitent les garanties de sécurité. Soit l'émetteur est anonyme (schéma N), soit l'échange perd la propriété de secret parfait (*Forward Secrecy*) car il repose sur des clés statiques (schémas K ou X). Pour obtenir à la fois l'authentification de l'émetteur et le secret parfait, Noise impose un handshake interactif, ce qui s'est relevé incompatible avec le système de relais et la nature asynchrone du réseau d'Idawi.

Nous avons donc réorienté notre choix vers NaCl et sa primitive CryptoBox. Cela nous a permis de concevoir un mécanisme de chiffrement authentifié sur mesure et non interactif : l'émetteur génère une clé éphémère qu'il signe avec sa clé privée permanente (statique), garantissant ainsi son authenticité auprès du destinataire. Dès lors qu'un participant connaît la clé publique d'un destinataire - celles-ci étant échangées de confiance hors réseau - il peut envoyer des messages chiffrés sans nécessiter d'échange préalable bloquant. Cette solution s'est révélée élégante pour notre cas d'utilisation.

Enfin, des contraintes de temps liées au calendrier du stage ont imposé de prioriser les développements et les intégrations. Il n'a pas été possible d'intégrer l'ensemble des fonctionnalités prévues dans la première version de Idawi. Ainsi, le chiffrement de la base de données via jfabe et le contrôle d'accès CP-ABE seront activés dans une seconde publication, la première version se contentant d'un stockage en clair. Ce compromis, dicté par la volonté de livrer rapidement un produit minimum viable, a néanmoins permis de concentrer les efforts sur la sécurisation des communications, qui constitue le socle indispensable des échanges dans de réseau, et de préparer soigneusement la montée en puissance des garanties de sécurité lors de la version suivante.

Enseignements et Apports du Stage

Ce stage a représenté une importante étape dans ma montée en compétences, en particulier dans le domaine de la cryptographie appliquée et la conception des architectures décentralisées. Avant de

rejoindre l'équipe, je ne connaissais pas l'existence du chiffrement basé sur les attributs ; j'ai découvert ce domaine de recherche fascinant grâce à ce stage. Cette expérience m'a offert l'opportunité de consulter et d'assimiler un volume conséquent de publications scientifiques et d'articles académiques. Cette immersion dans la littérature a constitué une chance d'apprentissage et a confirmé mon ambition professionnelle de poursuivre une carrière de chercheur.

Travailler aux côtés de chercheurs confirmés m'a également permis de mieux structurer mes explications et de formuler mes interrogations avec une plus grande précision. J'ai appris la nécessité d'envisager plusieurs pistes face à un même problème, en mesurant leurs implications théoriques et pratiques, tout en m'appuyant sur l'expertise, le regard critique et les conseils avisés de mes tuteurs. Cet encadrement m'a permis de prendre conscience de l'importance d'une architecture lisible, maintenable et évolutive.

Enfin, ce stage m'a offert la possibilité d'échanger avec d'autres chercheurs, d'établir des contacts et d'observer comment se construit une collaboration scientifique. J'en retire une motivation renforcée pour poursuivre dans cette voie, tout en mesurant avec enthousiasme le chemin qu'il me reste à parcourir.

CONCLUSION

Ce stage au sein du laboratoire I3S a été pour moi l'occasion de mettre en pratique ma passion pour la cryptographie dans un contexte de recherche appliquée exigeant. J'ai ainsi pu approfondir des domaines aussi variés que le chiffrement basé sur les attributs, les patrons de conceptions orientés objet, ainsi que le routage et les mécanismes de sécurisation des communications. Cette expérience a confirmé mon intérêt pour la recherche et mon souhait de poursuivre dans cette voie, en me donnant l'occasion de lire des publications scientifiques, d'échanger avec des chercheurs confirmés et d'établir des contacts précieux.

Le travail réalisé autour de la confidentialité des données locales et de la sécurisation des communications a montré qu'un *framework* décentralisé comme Idawi soulève des défis exigeants, qu'il s'agisse de la révocation des clés, de la synchronisation des états, du choix des algorithmes ou de la conception d'un protocole de communication sans échange préalable. Ces difficultés ont été traitées de manière pragmatique, en différant certaines fonctionnalités à une seconde version tout en livrant dès la première publication un socle de communication sûr et fonctionnel.

A l'issue de ce stage, je souhaite continuer à participer au projet Idawi en tant que contributeur externe, dans la mesure où cela sera possible. Je serais notamment intéressé à accompagner l'équipe dans l'intégration de la bibliothèque jfabe au code source du *framework*, à contribuer à sa maintenance et à son évolution, ainsi qu'à améliorer le schéma de communication fondé sur NaCl. Par ailleurs, j'aimerais, à un moment approprié, valoriser ces travaux en co rédigeant des articles avec Messieurs Dongo et Hogie, avec pour objectif direct une soumission au workshop sur les middlewares prévus en 2026.

Enfin, dans la continuité de cette Licence 3, je poursuivrai mes études dès le mois de septembre au sein du Master Parisien de Recherche en Informatique (MPRI) à l'Institut Polytechnique de Paris (École Polytechnique) à Palaiseau.

GLOSSAIRE

API (Application Programming Interface) :	Interface de programmation permettant à différents composants logiciels, comme l'interface graphique et la mémoire, de communiquer de façon standardisée.
Application décentralisée (Système distribué) :	Réseau informatique ou les données sont partagées entre plusieurs participants coopérant sans serveur central unique.
Attaque à chiffré choisi (CCA) :	Modèle d'attaque ou le pirate choisit délibérément des textes chiffres pour analyser le comportement du système lors du déchiffrement.
Attaque de préimage :	Tentative frauduleuse consistant à forger de fausses données pour produire une empreinte numérique identique à celle d'une donnée légitime.
Back-end :	Partie logicielle chargée de gérer la logique métier et le traitement interne des données de l'application.
CBOR :	Format de sérialisation de données binaire, plus compact et rapide à traiter que le JSON, idéal pour les transmissions réseau.
Chiffrement asymétrique :	Méthode cryptographique utilisant une paire de clés distinctes : une clé pour verrouiller l'information, et une autre clé pour la déverrouiller.
Chiffrement symétrique :	Méthode cryptographique utilisant une seule et même clé secrète à la fois pour chiffrer et déchiffrer des données.
Clé ephemere :	Clé cryptographique générée pour une courte durée (une session) puis détruite.
Couplage (Pairing) :	Opération mathématique avancée sur les courbes elliptiques permettant de relier deux groupes cryptographiques, fondamentale pour le fonctionnement du chiffrement par attributs.
Entropie :	Mesure du niveau d'aléatoire et d'imprévisibilité d'une information.
Fonction de hachage :	Algorithme transformant n'importe quelle donnée en une empreinte numérique unique, irréversible et de taille fixe.
Framework :	Infrastructure logicielle de base, fournissant un socle technologique sur lequel les développeurs s'appuient pour construire des applications.
Front-end :	Interface visuelle de l'application, correspondant à la partie avec laquelle l'utilisateur interagit directement.

Handshake interactif :	Phase de négociation technique nécessitant plusieurs allers-retours entre un émetteur et un destinataire avant de pouvoir établir une communication.
KEM / AEAD :	Le mécanisme d'encapsulation de clé (KEM) couplé à un chiffrement authentifié (AEAD) pour garantir simultanément la confidentialité et l'intégrité des données.
MVP (Minimum Viable Product) :	Première itération d'un logiciel déployé rapidement avec le strict minimum de fonctionnalités pour valider son utilité en conditions réelles.
Patron de conception (Design Pattern) :	Modèle d'architecture logicielle appliquée à des problèmes de code récurrents pour le rendre maintenable et évolutif.
Post-quantique :	Désigne les algorithmes de sécurité développés spécifiquement pour résister à la puissance de calcul des futurs ordinateurs quantiques.
Primitive cryptographique :	Opération mathématique de base agissant comme une brique pour construire des protocoles plus complexes.
Secret parfait (Forward Secrecy) :	Garantie assurant que la compromission de la clé privée d'un utilisateur ne permet pas de déchiffrer ses communications passées.
Sérialisation :	Procédé informatique consistant à transformer des objets complexes de la mémoire en octets pour le stockage ou le réseau.
Staging :	Environnement technique de test reproduisant les conditions réelles pour vérifier et valider une application avant son déploiement officiel en production.

BIBLIOGRAPHIE

Agrawal Shashank et Chase Melissa, FAME: Fast Attribute-based Message Encryption, Cryptology ePrint Archive (Paper 2017/807), 2017. URL: https://eprint.iacr.org/2017/807.

ANSSI, Appel à manifestation d'intérêt sécurité des écosystèmes de cybersécurité, Agence Nationale de la Sécurité des Systèmes d'Information, [En ligne], 2026. URL: https://cyber.gouv.fr/actualites/appel-a-manifestation-dinteret-securite-des-ecosystemes-de-cybersecurite.

Bellare Mihir, Desai Anand, Pointcheval David et Rogaway Phillip, Relations among notions of security for public-key encryption schemes, Cryptology ePrint Archive (Paper 1998/021), 1998. URL: https://eprint.iacr.org/1998/021.

Bernstein Daniel J., Lange Tanja et Schwabe Peter, The security impact of a new cryptographic library, Cryptology ePrint Archive (Paper 2011/646), 2011. URL: https://eprint.iacr.org/2011/646.

Bertoni Guido, Daemen Joan, Peeters Michaël et Van Assche Gilles, The Keccak Reference, [En ligne], 2011. URL: https://csrc.nist.gov/csrc/media/Projects/hash-functions/documents/Keccak-reference-3.0.pdf.

Bethencourt John, Sahai Amit et Waters Brent, Ciphertext-Policy Attribute-Based Encryption, 2007 IEEE Symposium on Security and Privacy (SP '07), 2007. URL: https://ieeexplore.ieee.org/document/4223236.

Boutrik Alexandre et Giron Alexandre Augusto, Geração de certificados KEM com ACME, Anais do XIV Seminário de Extensão e Inovação & XXIX Seminário de Iniciação Científica e Tecnológica da UTFPR - SEI/SICITE, Francisco Beltrão, UTFPR, 2024, [En ligne]. URL: https://www.even3.com.br/anais/seisicite2024/966085-GERACAO-DE-CERTIFICADOS-KEM-COM-ACME.

Boutrik Alexandre, jfabe : Fast Ciphertext-Policy Attribute-Based Encryption (CP-ABE) for Java, GitHub, [En ligne], 2026. URL: https://github.com/alexandreboutrik/jfabe.

Bowe Sean, BLS12-381: New zk-SNARK Elliptic Curve Construction, Electric Coin Company, [En ligne], 2017. URL: https://electriccoin.co/blog/new-snark-curve/.

Cintoo, Créer une synergie de données de scan 3D, [En ligne], 2026. URL: https://cintoo.com.

De Caro Angelo et Iovino Vincenzo, jPBC: Java pairing based cryptography, 2011 IEEE Symposium on Computers and Communications (ISCC), 2011, p. 850-855. URL: https://doi.org/10.1109/ISCC.2011.5983948.

Diffie Whitfield, Van Oorschot Paul C. et Wiener Michael J., Authentication and authenticated key exchanges, Designs, Codes and Cryptography, Berlin, Springer, 1992, p. 107-125.

EclEmma team, JaCoCo Java Code Coverage Library, [En ligne], 2009. URL: https://www.jacoco.org/jacoco/.

Eclipse Foundation, Eclipse IDE, [En ligne], 2001. URL: https://www.eclipse.org/.

EYENAV Robotics, EYENAV Robotics, [En ligne], 2026. URL: https://www.eyenavrobotics.com.

Galbraith Steven, Paterson Kenneth et Smart Nigel, Pairings for Cryptographers, Discrete Applied Mathematics, 156(16), 2008, p. 3113-3121. URL: https://doi.org/10.1016/j.dam.2007.12.010.

Goyal Vipul, Pandey Omkant, Sahai Amit et Waters Brent, Attribute-Based Encryption for Fine-Grained Access Control of Encrypted Data, Cryptology ePrint Archive (Paper 2006/309), 2006. URL: https://eprint.iacr.org/2006/309.

Hofheinz Dennis, Hövelmanns Kathrin et Kiltz Eike, A Modular Analysis of the Fujisaki-Okamoto Transformation, Cryptology ePrint Archive (Paper 2017/604), 2017. URL: https://eprint.iacr.org/2017/604.

Hogie Luc, Idawi: a decentralized middleware for achieving the full potential of the IoT, the fog, and other difficult computing environments, MIDDLEWEDGE 2022 - Proceedings of the 1st Workshop on Middleware for the Edge, 2022a, p. 1-5. URL: https://inria.hal.science/hal-03863333.

Hogie Luc, A Service-Oriented Middleware Enabling Decentralised Deployment in Mobile Multihop Networks, Lecture Notes in Computer Science, 2022b, p. 209-220. URL: https://inria.hal.science/hal-03886521.

Hogie Luc, A Decentralized Web Service Infrastructure for the Interoperability of Applications in Multihop Dynamic Networks, CIoT 2023 - 6th Conference on Cloud and Internet of Things, 2023, p. 211-218. URL: https://hal.science/hal-04075895.

Hogie Luc, How Digital Twins Can Improve the Design of Distributed Computing Frameworks, IEEE Xplore, 2025, p. 244-247. URL: https://hal.science/hal-05285529.

Hogie Luc, Byransha: A Simple Graph-Oriented Database for Java, GitHub, [En ligne], consulté en 2026. URL: https://github.com/lhogie/byransha.

Kim Taechan et Barbulescu Razvan, Extended Tower Number Field Sieve: A New Complexity for the Medium Prime Case, Cryptology ePrint Archive (Paper 2015/1027), 2015. URL: https://eprint.iacr.org/2015/1027.

Laboratoire I3S, Équipe Scale, Sophia Antipolis, Université Côte d'Azur / CNRS, [En ligne], 2026a. URL: https://scale.i3s.unice.fr/.

Laboratoire I3S, Pôle COMRED, Sophia Antipolis, Université Côte d'Azur / CNRS, [En ligne], 2026b. URL: https://www.i3s.unice.fr/fr/axes-de-recherche/comred/.

Laboratoire I3S, Robot Vision Team Homepage, [En ligne], 2026c. URL: https://webusers.i3s.unice.fr/robotvision/.

Liu Zhen, Huang Qiong et Wong Duncan S., On Enabling Attribute-Based Encryption to Be Traceable against Traitors, Cryptology ePrint Archive (Paper 2018/963), 2018. URL: https://eprint.iacr.org/2018/963.

Microsoft, Visual Studio Code: Code editing. Redefined., [En ligne], 2015. URL: https://code.visualstudio.com/.

MIRACL, MIRACL Core Cryptographic Library, GitHub, [En ligne], 2026. URL: https://github.com/miracl/core.

Neovim contributors, Neovim : Vim-fork focused on extensibility and usability, [En ligne], 2014. URL: https://neovim.io/.

Paar Christof, Pelzl Jan et Güneysu Tim, Understanding Cryptography: From Established Symmetric and Asymmetric Ciphers to Post-Quantum Algorithms, Second Edition, Berlin, Springer, 2024, p. 44.

Pearcode, Pearcode, Valbonne, [En ligne], 2026. URL: https://pearcode.io.

Perrin Trevor, The Noise Protocol Framework (Revision 34), [En ligne], 2018. URL: https://noiseprotocol.org/noise.html.

Riepel Doreen et Wee Hoeteck, FABEO: Fast Attribute-Based Encryption with Optimal Security, Cryptology ePrint Archive (Paper 2022/1415), 2022. URL: https://eprint.iacr.org/2022/1415.

Schmitz Gabriel, Boutrik Alexandre et Giron Alexandre Augusto, Towards Faster and Scalable Post-Quantum Hyperledger Fabric, Anais Estendidos do XXV Simpósio Brasileiro de Cibersegurança, Porto Alegre, SBC, 2025, p. 346-347. URL: https://sol.sbc.org.br/index.php/sbseg_estendido/article/view/41958.

Torvalds Linus et al., Git: Fast Version Control System, [En ligne], 2005. URL: https://git-scm.com/.

zkcrypto contributors, Issue #10: Serialization of Gt, GitHub, [En ligne], 2020. URL: https://github.com/zkcrypto/bls12_381/issues/10.

ANNEXES

ANNEXE 1 : Jfabe, Architecture Centrale et Contrat de Base

La figure présente l'architecture centrale de la bibliothèque définie dans le paquetage jfabe.core. L'interface ABEEngine y établit le contrat de base en agissant comme le socle d'un patron de conception Stratégie (*Strategy*). Cette approche permet de définir une interface uniforme pour les opérations cryptographiques et de rendre les différents schémas interchangeables sans impacter le reste du système, les implémentations concrètes de cette interface étant entièrement déléguées au paquetage jfabe.schemes.

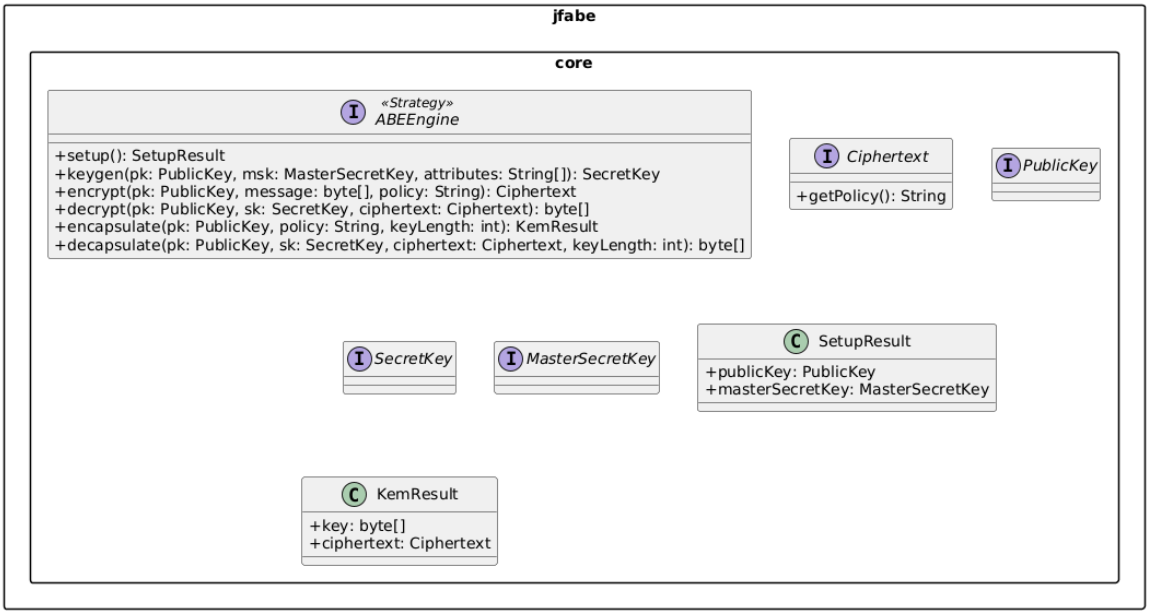


Figure 11 : Diagramme de classes illustrant le patron Stratégie pour le moteur central.[11]

[11] Source : Généré par l'auteur via PlantUML. Le code source est consultable à l'adresse suivante : https://github.com/alexandreboutrik/jfabe/blob/main/docs/diagrams/jfabe_core.

ANNEXE 2 : Jfabe, Interface Client

La figure ci-dessous détaille la conception de l'interface client située dans le paquetage jfabe.api. La classe JfabeClient fait office de patron Façade (*Facade*) pour masquer la complexité interne du moteur cryptographique. L'instanciation de ce client est rendue flexible grâce à l'utilisation du patron Monteur (*Builder*) qui gère la configuration et l'injection des dépendances. De plus, les énumérations Scheme, Transform et Feature implémentent le patron Fabrique (*Factory*) en associant chaque constante a une implémentation concrète du moteur.

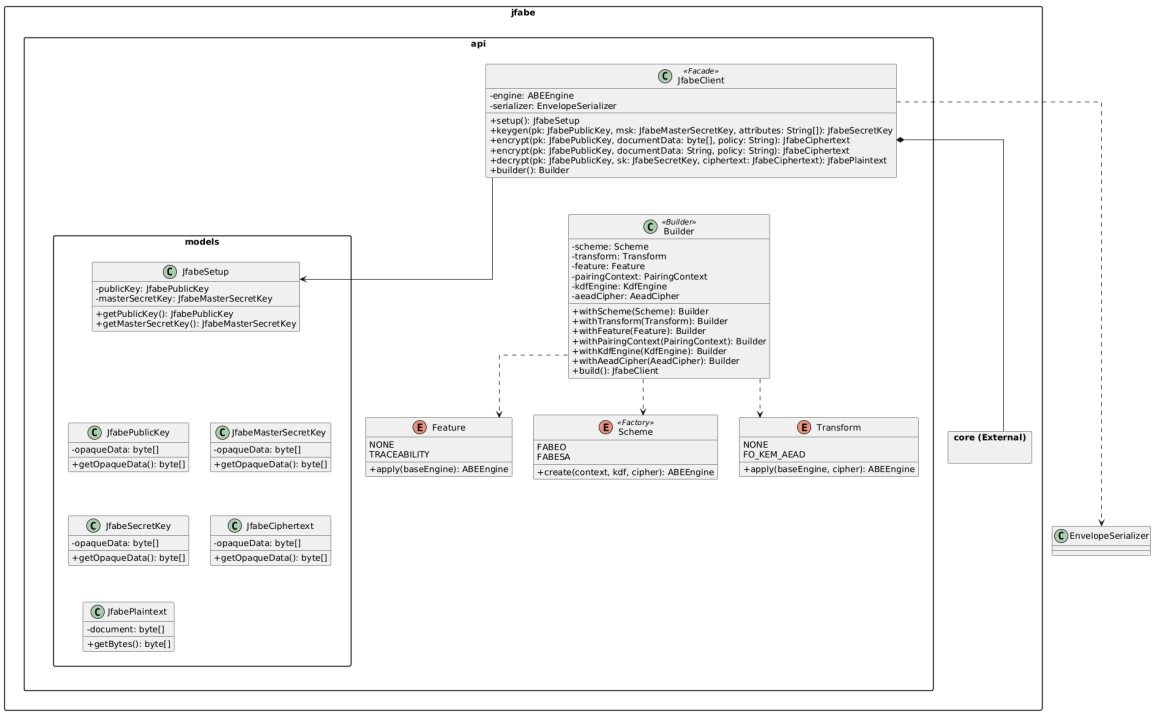


Figure 12 : Diagramme de classes de l'API client illustrant les patrons Façade, Monteur et Fabrique.[12]

[12] Source : Généré par l'auteur via PlantUML. Le code source est consultable à l'adresse suivante : https://github.com/alexandreboutrik/jfabe/blob/main/docs/diagrams/jfabe_api.

ANNEXE 3 : Jfabe, Implémentation Cryptographique

La figure ci-dessous illustre l'implémentation cryptographique spécifique contenue dans les paquetages jfabe.schemes. La classe FabeoEngine y intervient en tant que stratégie concrète (*Strategy*) en implémentant l'interface ABEEngine afin de fournir la logique métier propre au schéma FABEO.

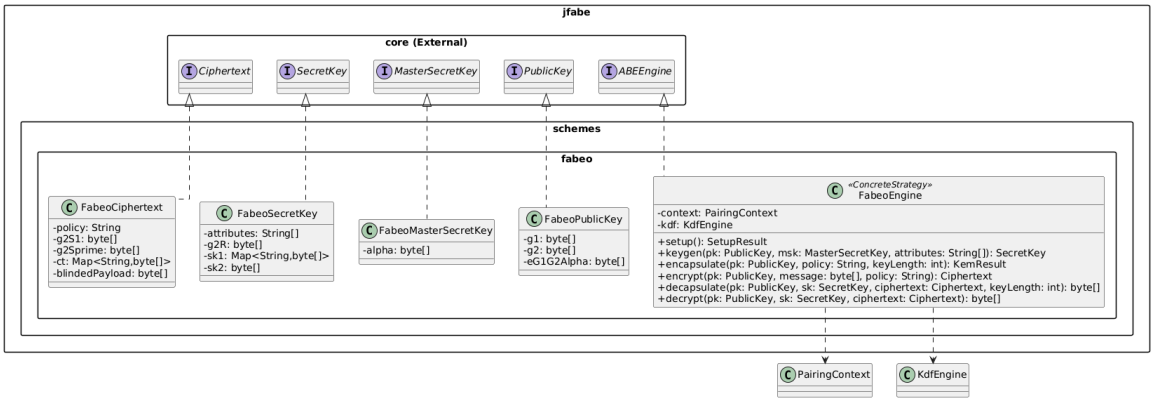


Figure 13 : Diagramme de classes détaillant la stratégie concrète du schéma FABEO.[13]

[13] Généré par l'auteur via PlantUML. Le code source est consultable à l'adresse suivante : https://github.com/alexandreboutrik/jfabe/blob/main/docs/diagrams/jfabe_schemes.

ANNEXE 4 : Jfabe, Transformations Cryptographiques

La figure ci-dessous expose l'application des transformations cryptographiques gérées dans jfabe.transform, constituant une illustration du patron Décorateur (*Decorator*). La classe FoKemAeadDecorator enveloppe le moteur ABEEngine de base pour lui ajouter de manière transparente une sécurité CCA via la transformation de Fujisaki-Okamoto combinée à un chiffrement symétrique AEAD. Cette architecture permet d'améliorer la sécurité et de lever les limites de taille des documents (*input*), sans jamais modifier le code source du schéma cryptographique sous-jacent.

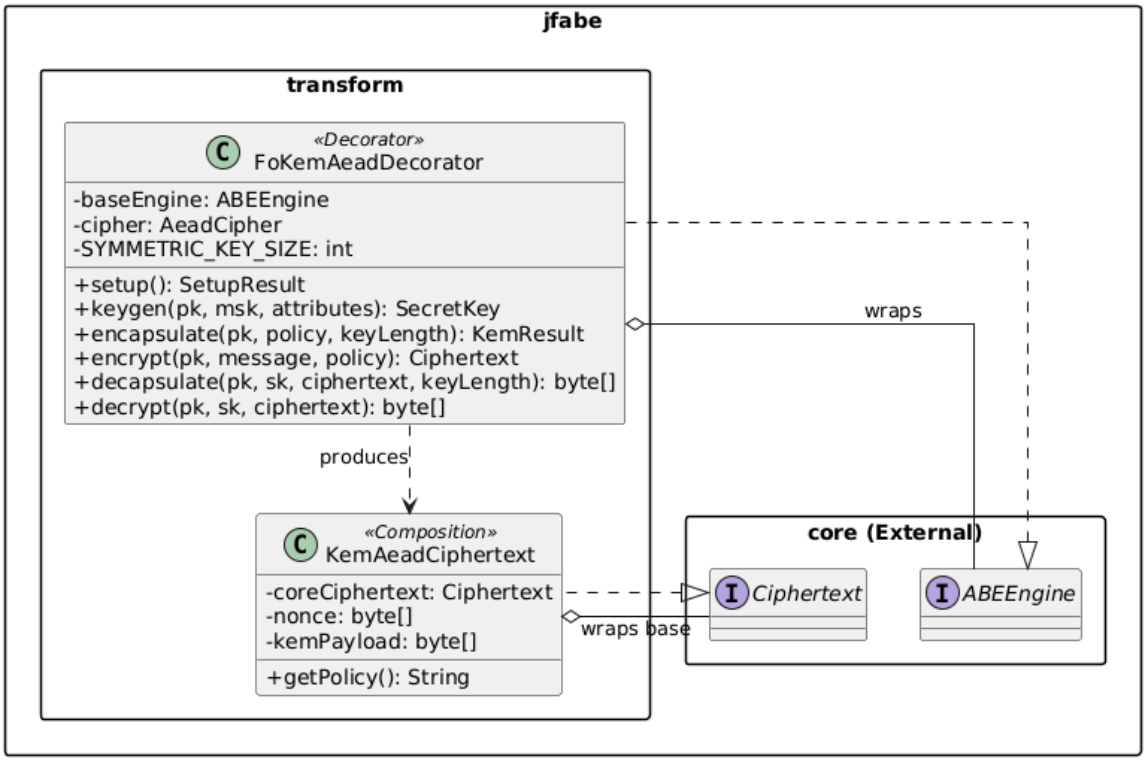


Figure 14 : Diagramme de classes du patron Décorateur implémentant Transform.FO_KEM_AEAD.[14]

[14] Généré par l'auteur via PlantUML. Le code source est consultable à l'adresse suivante : https://github.com/alexandreboutrik/jfabe/blob/main/docs/diagrams/jfabe_transform.

ANNEXE 5 : Jfabe, Abstraction Mathématique

La figure ci-dessous démontre comment le paquetage jfabe.pairing exploite le patron Adaptateur (*Adapter*) afin d'éviter un couplage fort entre la bibliothèque et la dépendance cryptographique externe MiraCL. La classe MiraclBls12381Context adapte ainsi les fonctionnalités de la bibliothèque MiraCL pour qu'elles correspondent à l'interface interne PairingContext. Le choix garantit que le reste du code demeure totalement indépendant de la bibliothèque de courbes elliptiques sous-jacente.

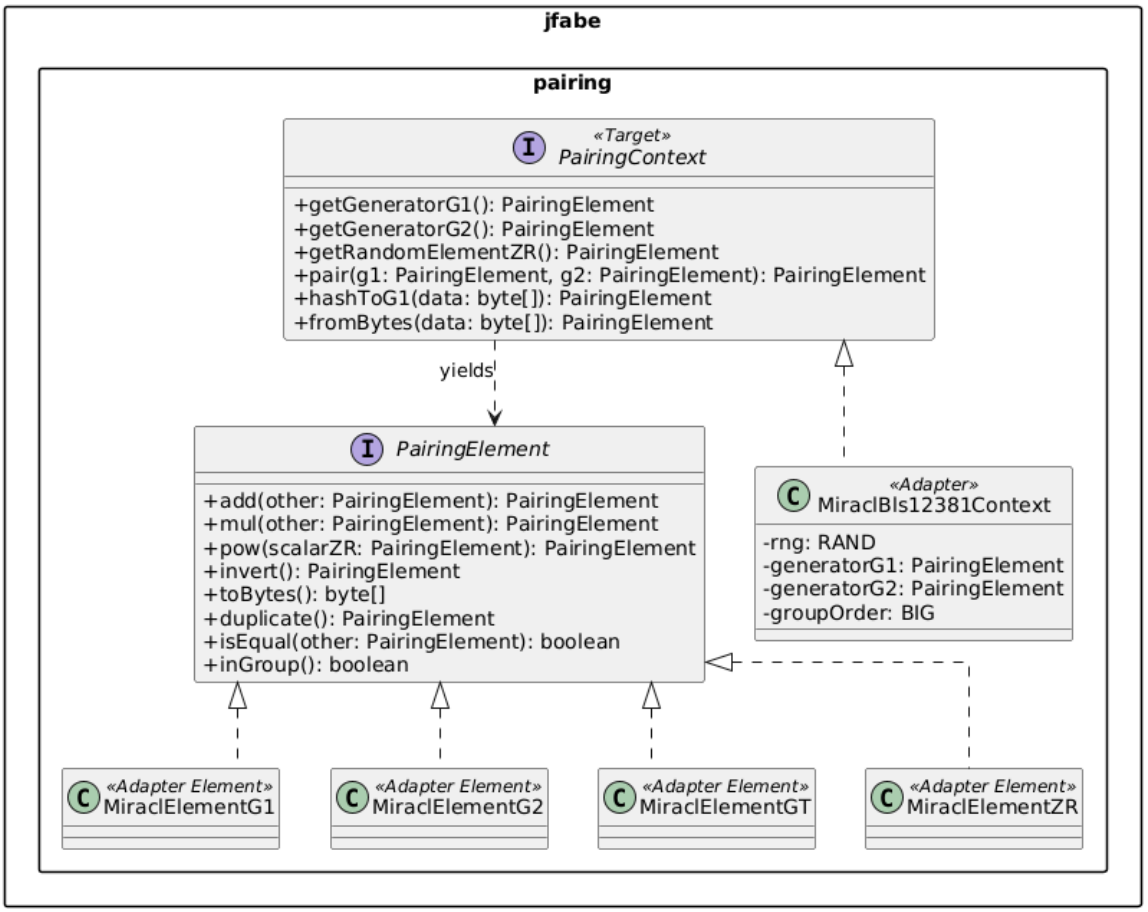


Figure 15 : Diagramme de classes du patron Adaptateur pour les dépendances externes.[15]

[15] Source : Généré par l'auteur via PlantUML. Le code source est consultable à l'adresse suivante : https://github.com/alexandreboutrik/jfabe/blob/main/docs/diagrams/jfabe_pairing.

ANNEXE 6 : Jfabe, Analyse des politiques d'accès

La figure ci-dessous se concentre sur le traitement et l'analyse des politiques d'accès dans jfabe.policy, qui reposent sur la construction d'un arbre syntaxique abstrait modélisé selon le patron Composite (*Composite*). La structure combine l'interface AccessPolicy avec des nœuds de type AttributeNode, AndNode et OrNode pour représenter fidèlement l'arbre d'accès booléen.

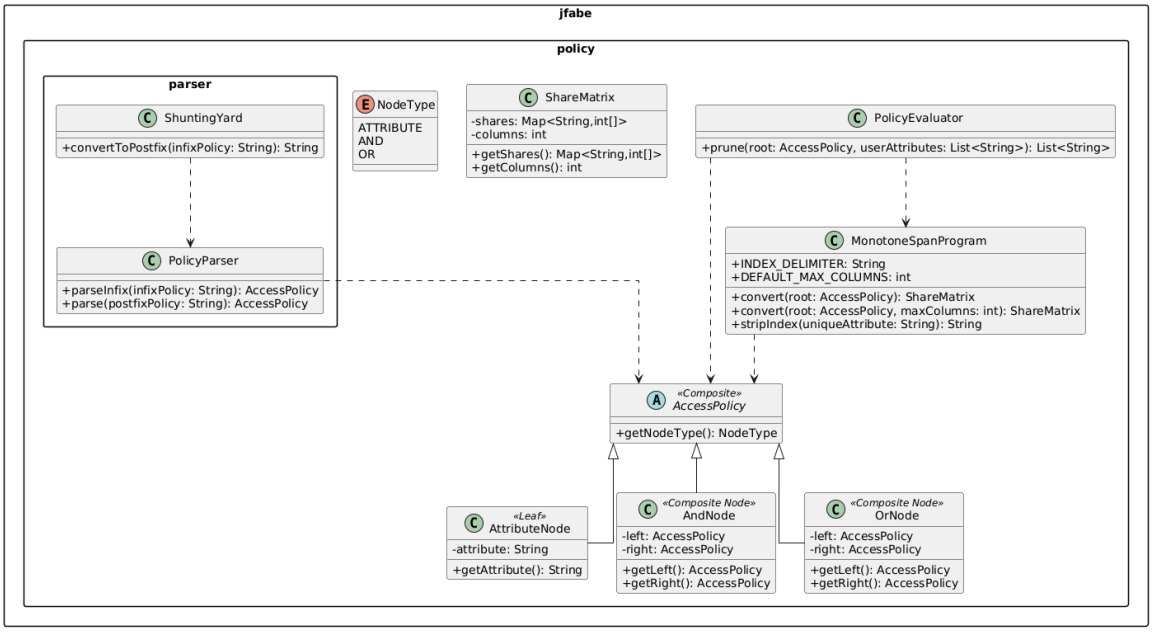


Figure 16 : Diagramme de classes illustrant le patron Composite pour la modélisation de l'arbre syntaxique des politiques d'accès.[16]

[16] Source : Généré par l'auteur. Le code source est consultable à l'adresse suivante : https://github.com/alexandreboutrik/jfabe/blob/main/docs/diagrams/jfabe_policy.

ANNEXE 7 : Jfabe, Sérialisation des données

La figure ci-dessous schématise la gestion des données destinées à transiter sur le réseau, assurée par le paquetage jfabe.serialization. La classe EnvelopeSerializer y convertit les objets cryptographiques en enveloppes binaires opaques au format CBOR. Cela garantit que les clés et les textes chiffrés sont correctement traduit en tableaux d'octets de manière compacte pour les échanges réseau.

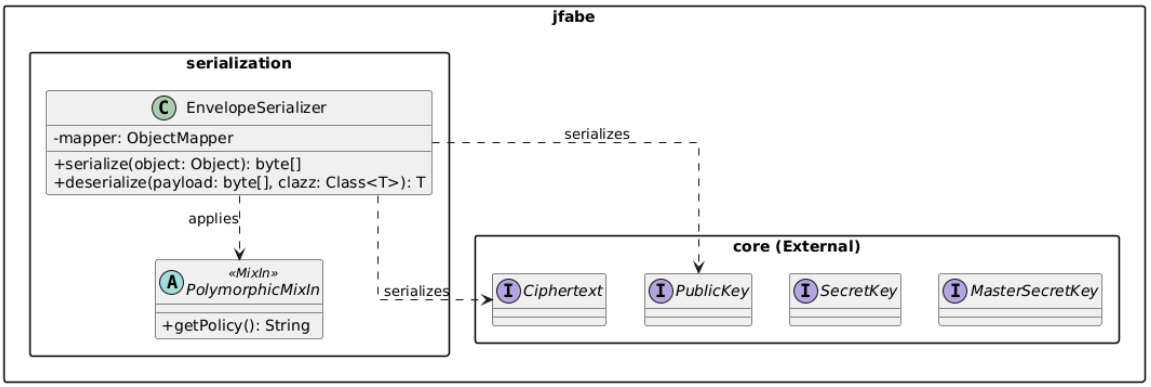


Figure 17 : Diagramme de classes détaillant le processus de sérialisation CBOR.[17]

[17] Source : Généré par l'auteur. Le code source est consultable à l'adresse suivante : https://github.com/alexandreboutrik/jfabe/blob/main/docs/diagrams/jfabe_serialization.

ANNEXE 8 : Jfabe, Exemple d'Utilisation de l'API

Cet extrait du code illustre comment un client interagit avec l'API de jfabe. Il démontre la configuration du moteur cryptographique, la génération des clés, ainsi que le processus complet de chiffrement et de déchiffrement d'un document.

```
import io.github.alexandreboutrik.jfabe.api.JfabeClient;
import io.github.alexandreboutrik.jfabe.api.models.*;
import io.github.alexandreboutrik.jfabe.api.enums.*;
import io.github.alexandreboutrik.jfabe.exceptions.*;

JfabeClient client = JfabeClient.builder()
    .withScheme(Scheme.FABEO)
    .withTransform(Transform.FO_KEM_AEAD)
    .build();

String[] keyAttrList = {"Team_A", "Team_B"};
String policy = "(Team_A AND Team_B) OR Team_C";
String documentData = "This is the document's data";

JfabeSetup setup = client.setup();
JfabePublicKey pk = setup.getPublicKey();
JfabeMasterSecretKey msk = setup.getMasterSecretKey();

JfabeSecretKey sk = client.keygen(pk, msk, keyAttrList);
JfabeCiphertext encryptedDocument = client.encrypt(pk, documentData, policy);

try {
    JfabePlaintext decryptedDocument = client.decrypt(pk, sk, encryptedDocument);
    byte[] recoveredDocument = decryptedDocument.getBytes();
} catch (UnsatisfiedPolicyException e) {
    // User's attributes do not satisfy the document's policy.
} catch (CiphertextIntegrityException e) {
    // Ciphertext has been tampered with or corrupted.
}
```